

Лабораторная работа №7

«Работа с базой данных с помощью библиотеки SQLAlchemy. Работа с системой контроля версий Git»

Цель работы: Добавление функционала извлечения тестируемых изображений и результатов классификации из базы данных. Реализация пользовательского интерфейса отображения результатов классификации. Работа с системой контроля версий Git.

Системой контроля версий “Git” называют программное обеспечение, предназначенное для контроля различных версий файлов. Назначение этой системы отслеживать изменения, сделанные в файлах и, при необходимости, возвращаться к предыдущим версиям. Т.е. эта система позволяет фиксировать состояние файлов в определенные моменты времени. Эта система позволяет вернуться к предыдущей версии файла при необходимости сделать это.

Для начала работы с данной системой контроля версий ее сначала необходимо установить на Ваш компьютер. Для этого перейдите по ссылке <https://git-scm.com/install/> и выберите установку в зависимости от используемой операционной системы. Например для Windows (рисунок 1).

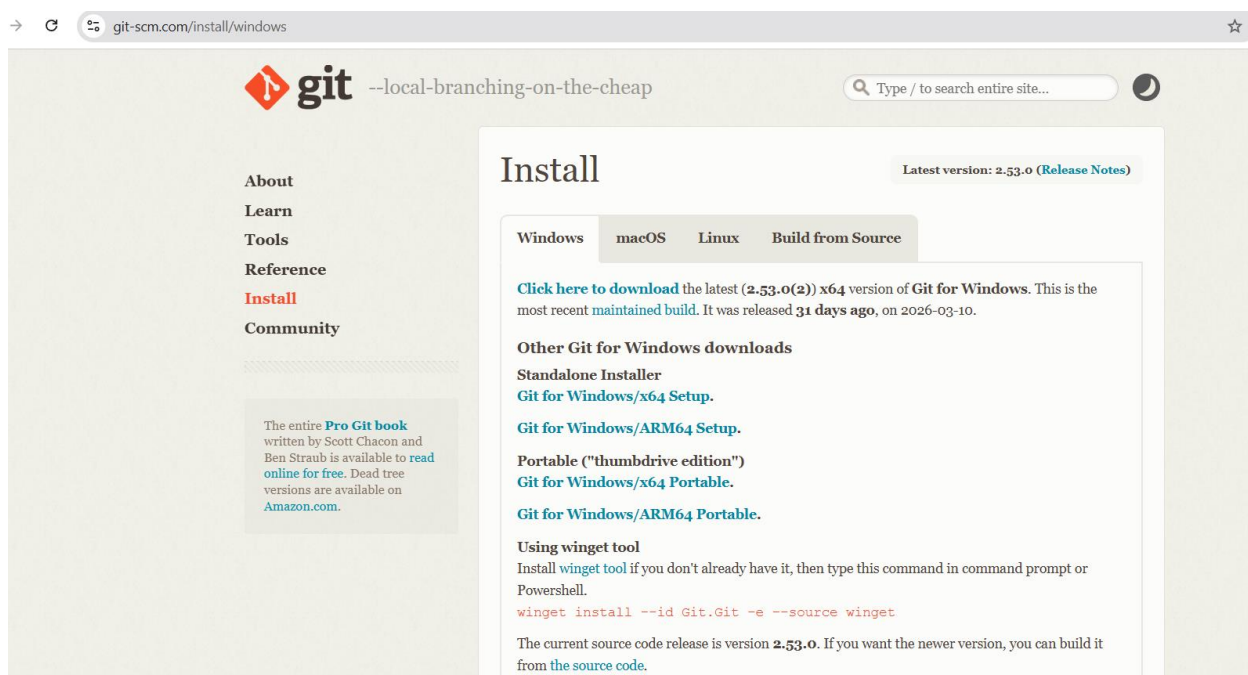


Рисунок 1. – Установка Git

Если у Вас «Git» уже установлен, то эту часть работы, связанную с установкой и настройкой «Git» можно пропустить.

Система «Git» позволяет выполнить настройку, которая включает в себя имя пользователя, его адрес электронной почты и редактор кода. Данная информация затем будет отражаться в каждой сохраненной версии проекта (в каждом Commit) системой «Git».

Для работы с конфигурацией необходимо использовать утилиту Git config.

Различают три уровня конфигурации:

1. Конфигурация, покрывающая всех пользователей системы;

```
git config --system
```

2. Конфигурация, которая будет применяться к конкретному пользователю системы контроля версий. Она будет применяться ко всем репозиториям данного пользователя. Реализуется при помощи команды:

```
git config --global
```

3. Конфигурация, применяемая к текущему репозиторию, в котором находится проект (файлы), версии которых необходимо контролировать. Реализуется с помощью команды:

```
git config
```

или

```
git config --local
```

После установки системы контроля версий «Git» откройте терминал и выполните настройку глобальной конфигурации для username и электронной почты выполнив команды ниже:

```
git config --global user.name "Ваше имя пользователя"
```

```
git config --global user.email "youremail@yandex.ru"
```

где в кавычках должны быть соответственно указано Ваше имя пользователя, например «Roman» (для первой команды), а для второй команды необходимо указать адрес Вашей электронной почты в кавычках.

Для создания репозитория в котором будет осуществляться контроль версий, размещенных в нем файлов предусмотрена команда

```
git init
```

После выполнения данной команды будет создана скрытая папка `.git` в которой будут содержаться различные версии файлов.

Теперь данная директория (папка) является репозиторием.

Если теперь создать внутри данного репозитория какой-либо файл, то при помощи команды

```
git log
```

можно проверить историю изменений версий файлов, находящихся в репозитории.

Если никаких изменений с файлами в репозитории не происходило, то на консоль выведется следующее сообщение:

```
[Romans-MacBook-Air:gittut romankovalchik$ git log
fatal: your current branch 'master' does not have any commits yet
Romans-MacBook-Air:gittut romankovalchik$ █
```

Еще одной командой, позволяющей отслеживать состояние файлов в репозитории является команда

```
git status
```

Реализация данной команды к файлам, с которыми не выполняли никаких изменений приведет к результату, приведенному ниже:

```
[Romans-MacBook-Air:gittut romankovalchyk$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)

        app.py

nothing added to commit but untracked files present (use "git add" to track)
```

Рисунок 2.

Файлы, находящиеся в репозитории (tracked) могут быть под версионным контролем и не под версионным контролем (untracked).

Если у Вас уже создан локальный репозиторий, то после выполнения команды `git log` Вы увидите сообщение по примеру, приведенному на рисунке 3. Оно говорит о том, что Ваш проект находится под версионным контролем системой Git

```
(.venv) PS C:\Users\Professional\Desktop\HackRF\CV> git log
commit f0eaf9d589178a2181e50ab4c29eca382dd5b5ac (HEAD -> develop, master)
Author: RomanKovalchik <kovalchykrv@yandex.ru>
Date:   Mon Mar 30 04:00:27 2026 +0300

    Initial commit

(.venv) PS C:\Users\Professional\Desktop\HackRF\CV>
```

Рисунок 3.

В противном случае, если файлы проекта не помещены под версионный контроль, то при выполнении команды `git log` в терминал будет выведен следующий результат:

```
(.venv) PS C:\Users\Professional\Desktop\HackRF\PythonGit> git log
fatal: not a git repository (or any of the parent directories): .git
(.venv) PS C:\Users\Professional\Desktop\HackRF\PythonGit>
```

Рисунок 4

Если репозиторий создан, но при этом не создано ни одного commit, то в консоль будет выведено следующее сообщение, показанное на рисунке 5.

```
(.venv) PS C:\Users\Professional\Desktop\HackRF\PythonGit> git log
fatal: your current branch 'master' does not have any commits yet
```

Рисунок 5.

Ход работы: Откройте проект, в котором реализовано веб-приложение для классификации геометрических фигур. Создайте в корневой директории проекта файл с названием «.gitignore» как показано на рисунке 6.

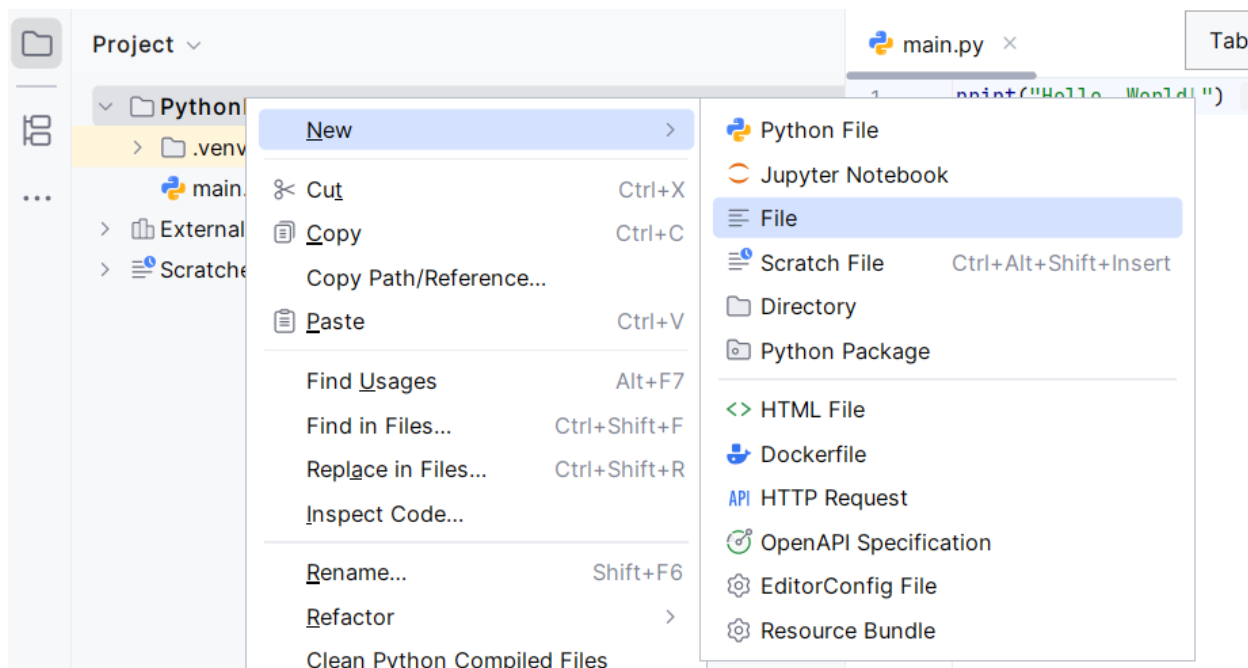


Рисунок 6. – Создание файла «.gitignore»

Поместите в файл «.gitignore» следующие файлы:

```
.venv/ *  
.pyc __pycache__/  
instance/  
.pytest_cache .coverage htmlcov/  
dist/ build/ *.egg-info/
```

Файл «.gitignore» предназначен для того чтобы указать какие файлы НЕ будут находиться под версионным контролем и не будут потом добавлены в удаленный репозиторий.

Затем выполните команду для помещения проекта под версионный контроль:

```
git init
```

Она, как было указано выше, необходима чтобы создать репозиторий

Затем выполните команду для подготовки файлов к фиксации версий

`git add .`

После чего выполните команду

`git commit -m "Initial commit"`

Этой командой фиксируются версии файлов, и коммиту дается название, например «Initial commit»

Для того чтобы просмотреть историю фиксаций, в которой должен быть пока только один commit с названием "Initial commit" выполните в терминале команду:

`git log`

В результате в терминале должны быть выполнены следующие команды и показан результат их выполнения (рисунок 7)

```
(.venv) PS C:\Users\Professional\Desktop\HackRF\PythonProject3> git init
Initialized empty Git repository in C:/Users/Professional/Desktop/HackRF/PythonProject3/.git/
(.venv) PS C:\Users\Professional\Desktop\HackRF\PythonProject3> git add .
warning: in the working copy of '.idea/inspectionProfiles/Project_Default.xml', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.idea/inspectionProfiles/profiles_settings.xml', LF will be replaced by CRLF the next time Git touches it
(.venv) PS C:\Users\Professional\Desktop\HackRF\PythonProject3> git commit -m "Initial commit"
[master (root-commit) 6284790] Initial commit
 9 files changed, 81 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 .idea/.gitignore
 create mode 100644 .idea/PythonProject3.iml
 create mode 100644 .idea/inspectionProfiles/Project_Default.xml
 create mode 100644 .idea/inspectionProfiles/profiles_settings.xml
 create mode 100644 .idea/misc.xml
 create mode 100644 .idea/modules.xml
 create mode 100644 .idea/vcs.xml
 create mode 100644 main.py
(.venv) PS C:\Users\Professional\Desktop\HackRF\PythonProject3> git log
commit 62847904715afd54253399d01e9b703a87977e3b (HEAD -> master)
Author: RomanKovalchik <kovalchykrv@yandex.ru>
Date:   Fri Apr 10 04:11:29 2026 +0300

    Initial commit
(.venv) PS C:\Users\Professional\Desktop\HackRF\PythonProject3>
```

Рисунок 7.

Данное задание необходимо выполнить в новой ветке проекта (проект находится под версионным контролем). Назовите новую ветку "develop".

Создать новую ветку можно при помощи команды:

`git branch develop`

Для перехода в новую ветку необходимо выполнить команду:

`git checkout <branch's name>`

В нашем случае это будет команда:

`git checkout develop`

Наряду с этим, создание новой ветки и переход в нее можно выполнить при помощи одной команды Git. В нашем случае это будет команда:

`git checkout -b develop`

После этого Вы можете выполнить команду

`git log`

Дальнейшая разработка приложения будет осуществляться в ветке «develop».

Следующим этапом необходимо добавить возможность отображать историю классификаций объектов на цифровых изображениях.

Отображение результатов классификации объектов будет выполняться с использованием отдельного шаблона, создайте его в пакете «templates» назовите “history.html”. Затем создайте в контроллере отдельный эндпоинт, который будет возвращать данный шаблон со списком, каждым элементом которого будет являться информация о конкретном результате классификации изображения. В данном эндпоинте будет выполняться запрос в базу данных с помощью которого будут извлекаться все данные из таблицы «Predictions», в которой хранятся результаты классификации.

Полезным инструментом для дебагинга, для того чтобы просмотреть какие таблицы находятся в базе данных, является метод inspect. Для того, чтобы им воспользоваться импортируйте его сначала из sqlalchemy при помощи кода:

```
from sqlalchemy import inspect
```

также импортируйте engine:

```
from database.database import db_session, init_db, engine
```

Теперь в эндпоинт в который будут выбираться данные из базы поместите код:

```
@app.route('/history', methods=['GET', 'POST'])
def history():
    from sqlalchemy import inspect
    inspector = inspect(engine)
    print(inspector.get_table_names())
    predictions = None
    return render_template('history.html', predictions=predictions)
```

Если все работает корректно, то в терминале будет информация о таблицах базы данных, в данном случае эта информация приведена ниже:

```
['Predictions', 'alembic_version']
```

В дальнейшем нам этот код не потребуется, его можно удалить или закомментировать. Это полезный инструмент для дебагинга, когда есть необходимость выяснить есть ли соединение с базой данных.

Одна из возможных реализаций эндпоинта для отображения данных о классификации может быть такой:

```
@app.route('/history', methods=['GET'])
def history():
    stmt = select(Predictions)
    stmt = stmt.order_by(Predictions.id.desc())
    predictions = db_session.execute(stmt).scalars().all()
    db_session.commit()
    return render_template('history.html', predictions=predictions)
```

Ниже приведен один из возможных вариантов реализации шаблона «history.html», который предназначен для отображения результатов классификации

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>История предсказаний</title>
</head>
<body>
  <h1>История предсказаний</h1>
  <a href="/">← На главную</a>

  {% if predictions %}
    <table border="1" cellpadding="10" cellspacing="0">
      <thead>
        <tr>
          <th>ID</th>
          <th>Изображение</th>
          <th>Фигура</th>
          <th>Уверенность</th>
          <th>Вероятность круга</th>
          <th>Вероятность квадрата</th>
          <th>Вероятность треугольника</th>
          <th>Имя файла</th>
          <th>Дата создания</th>
        </tr>
      </thead>
      <tbody>
        {% for prediction in predictions %}
          <tr>
            <td>{{ prediction.id }}</td>
            <td>
              
            </td>
            <td>
              {% if prediction.predicted_class == 'circle' %}Круг
              {% elif prediction.predicted_class == 'square' %}Квадрат
              {% else %}Треугольник{% endif %}
            </td>
            <td>{{ "%.1f"|format(prediction.confidence * 100) }}%</td>
            <td>{{ "%.1f"|format(prediction.proBABILITIES_circle * 100) }}%</td>
            <td>{{ "%.1f"|format(prediction.proBABILITIES_square * 100) }}%</td>
            <td>{{ "%.1f"|format(prediction.proBABILITIES_triangle * 100) }}%</td>
            <td>{{ prediction.filename }}</td>
            <td>{{ prediction.created_at.strftime('%d.%m.%Y %H:%M:%S') if prediction.created_at }}</td>
          </tr>
        {% endfor %}
      </tbody>
    </table>
  {% else %}
    <p>Пока нет предсказаний. Загрузите изображение на главной странице.</p>
  {% endif %}
```

```
</body>
</html>
```

Замените код, находящийся в файле «history.html» на код приведенный выше, или реализуйте свое решение для вывода результатов классификации объектов.

Как следует и приведенного кода, в данном случае результаты классификации представлены в виде таблицы, в каждой строке которой находятся результаты конкретного анализа.

Обратите на данную данный код представления, разберитесь как он работает:

```
<td>
    
</td>
```

В данном приложении бинарное цифровое изображение в виде строки в базе данных. Это не единственный способ сохранения изображений в базе данных, существуют и другие способы. Для возможности отображения в браузере необходимо строковое представление изображения конвертировать обратно в бинарный формат. Это необходимо сделать со всеми изображениями. Для реализации конвертации создадим в контроллере создадим эндпоинт который будет вызываться для каждого изображения, в эндпоинт будет передаваться ID результата классификации, они тоже хранятся в базе данных. Реализация данного эндпоинта приведена ниже. Добавьте его в контроллер.

```
@app.route('/image/<int:prediction_id>')
def get_image(prediction_id):
    #Возвращает изображение по ID предсказания
    prediction = db_session.query(Predictions).get(prediction_id)
    if prediction and prediction.image_data:
        # Декодируем base64 в байты
        image_bytes = base64.b64decode(prediction.image_data)
        return send_file(
            io.BytesIO(image_bytes),
            mimetype='image/png',
            as_attachment=False
        )
    return "Image not found", 404
```

Как видим, метод будет возвращать изображение в атрибут «src», что сделает возможным отображение изображения в браузере.

После эндпоинта выше также добавьте в контроллер метод для закрытия сессии работы с базой данных:

```
# Закрывать сессию
@app.teardown_appcontext
def shutdown_session(exception=None):
    db_session.remove()
```

Также необходимо добавить соответствующую гипертекстовую ссылку или кнопку для перехода на страницу с историей классификации. Она должна быть добавлена в шаблон «index.html». Возможная реализация ссылки приведена ниже:


```
<a href="/history" class="btn btn-secondary" style="text-decoration: none; display: inline-block; text-align: center;">
```

История классификаций

```
</a>
```

В качестве дополнительного задания можете реализовать функционал удаления результатов классификации либо сортировке по дате.

Зафиксируйте сделанные в проекте изменения

Для этого выполните в терминале команду:

```
git add .
```

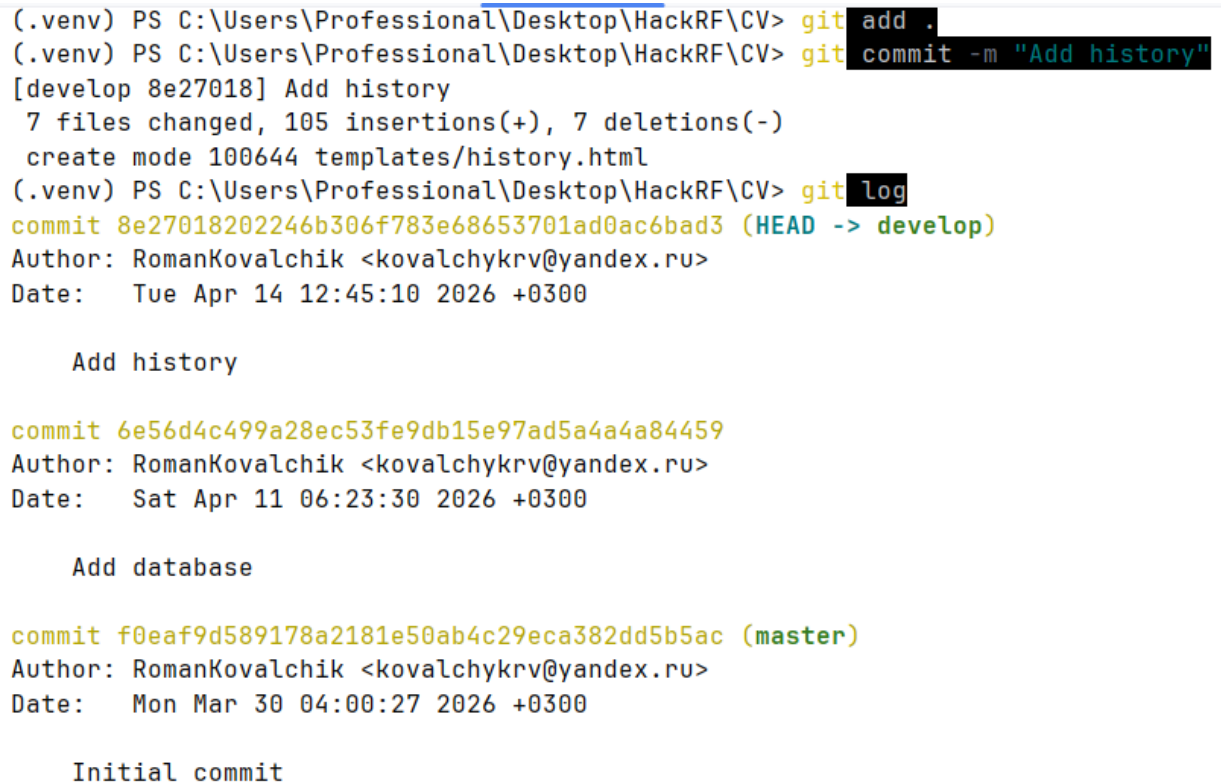
и команду:

```
git commit -m "Add history"
```

Затем выведите историю commits при помощи команды:

```
git log
```

Если все сделано корректно, то в терминале должно появиться сообщение как на рисунке



```
(.venv) PS C:\Users\Professional\Desktop\HackRF\CV> git add .
(.venv) PS C:\Users\Professional\Desktop\HackRF\CV> git commit -m "Add history"
[develop 8e27018] Add history
 7 files changed, 105 insertions(+), 7 deletions(-)
 create mode 100644 templates/history.html
(.venv) PS C:\Users\Professional\Desktop\HackRF\CV> git log
commit 8e27018202246b306f783e68653701ad0ac6bad3 (HEAD -> develop)
Author: RomanKovalchik <kovalchykrv@yandex.ru>
Date: Tue Apr 14 12:45:10 2026 +0300

    Add history

commit 6e56d4c499a28ec53fe9db15e97ad5a4a84459
Author: RomanKovalchik <kovalchykrv@yandex.ru>
Date: Sat Apr 11 06:23:30 2026 +0300

    Add database

commit f0eaf9d589178a2181e50ab4c29eca382dd5b5ac (master)
Author: RomanKovalchik <kovalchykrv@yandex.ru>
Date: Mon Mar 30 04:00:27 2026 +0300

    Initial commit
```

Рисунок 1. – История коммитов

Подумайте, для каких практических целей можно применить такое или похожее веб-приложение работающее с анализом цифровых изображений.